



Introduction to TinyML: Running Deep Learning Models on low-power Micro-Controllers

Tutorial on Embedded World Conference 2025

Daniel Mueller-Gritschneider, TU Vienna

Marcus Rüb, Hahn-Schickard Institut

Philipp van Kempen, TU Munich

Presenters

- **Daniel Müller-Gritschneider**
Embedded Computing Systems, TU Wien
- **Marcus Rüb**, Hahn-Schickard Institut,
TinyML Germany, AI Trainer
- **Philipp van Kempen**, TU Munich
Research on TinyML Methods, Apache TVM & RISC-V

TinyML Tutorial Overview

14:00 Daniel Müller-Gritschneider:
Introduction to TinyML

14:45 Break and Q&A

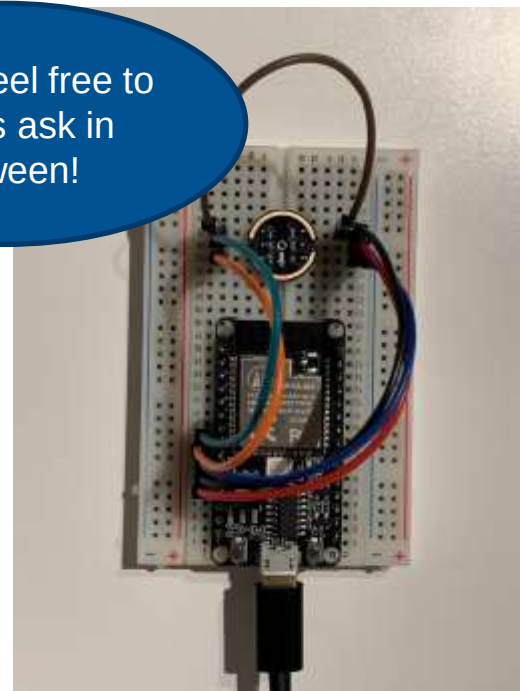
15:00 Marcus Rüb:
Introduction to Model Compression Techniques

15:30 Break and Q&A

15:45 Philipp van Kempen:
Showcase: TinyML Keyword Spotting (KWS)

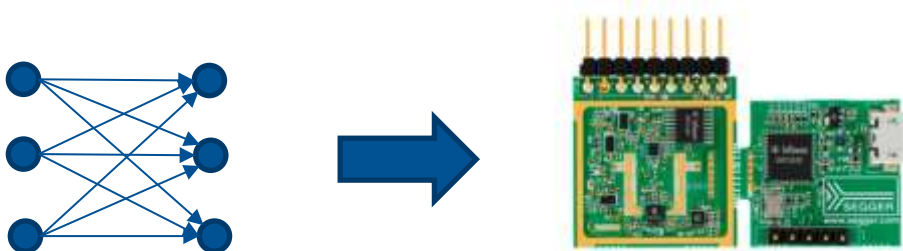
16:30 Q&A and Wrap-up

Please feel free to
always ask in
between!



Goal of this Tutorial

Give you an introduction and getting started in the field of *tinyML*



Machine
Learning

ML Compilers

Embedded SW

Embedded HW

Machine
Learning

Who already worked with ML frameworks such as TensorFlow, PyTorch or similar?

ML Compilers

Who already worked with ML Compilers such as TFLite, TVM, IREE or similar?

Embedded SW

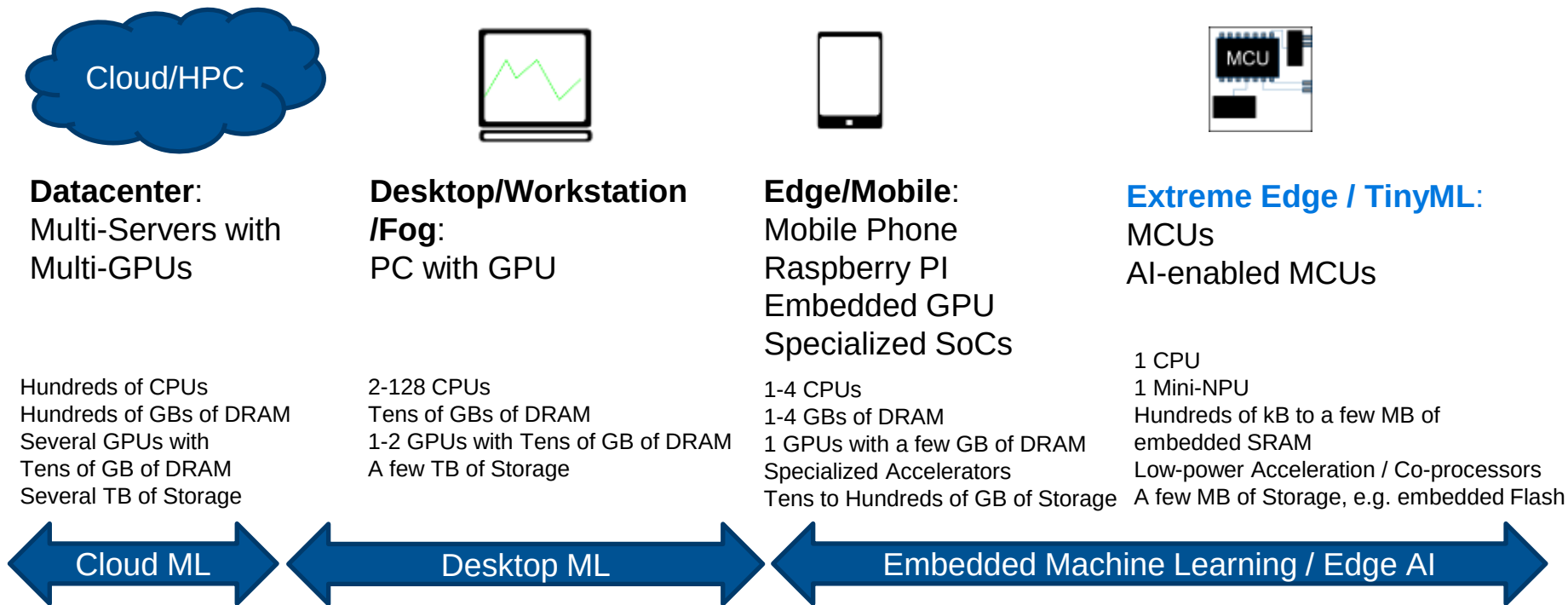
Who already programmed MCUs?

Embedded HW

ML Models are Heterogeneous

- Large language models (LLMS) GPT-4, ChatGPT: ~1.7 trillion parameters - produces human-like text
- YOLOv8 – millions of parameters – Image classification e.g. for autonomous driving
- Keyword Spotting KWS: 16k-300k parameters – Detects keyword in an audio stream, e.g. for Audio wakeup

ML Platforms are Heterogeneous



Embedded Machine Learning / Edge AI

- Advantages:

- Uplink Bandwidth reduction
- Latency reduction
- Autonomy (smart devices)
- Privacy (Data stays on the device)
- Energy (no expensive data transfers)

- Challenges:

- Edge devices are resource-constrained
- Development of embedded ML applications requires a unique skillset

Some EdgeAI Applications Domains

- Smart Home
- Surveillance
- Autonomous Robots
- Healthcare Monitoring
- Predictive Maintenance
- Smart Agriculture

Sensors

How do we make these devices smart?



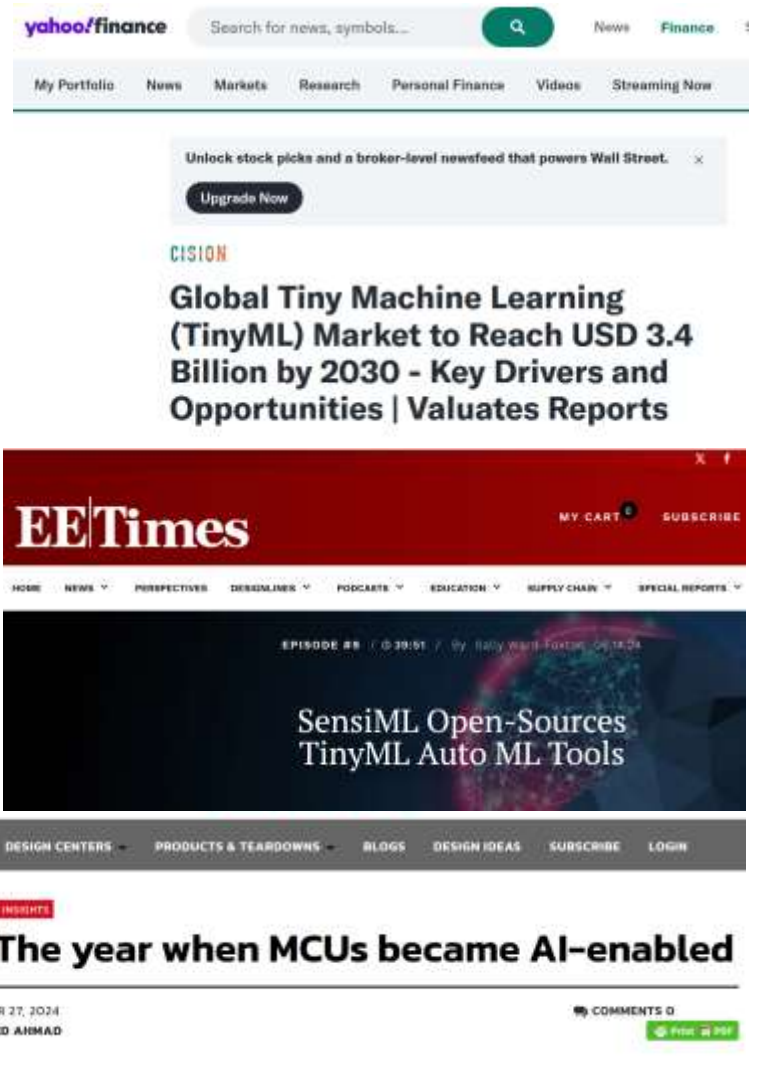
TinyML

- Running ML on low-power MCUs
- Significant progress in design of tinyML applications development (AutoML)
- AI-enabled Micro-Controller Units (MCUs) becoming available

<https://www.edn.com/2024-the-year-when-mcus-became-ai-enabled/>

<https://finance.yahoo.com/news/global-tiny-machine-learning-tinyml-153000910.html>

<https://www.eetimes.com/podcasts/sensiml-open-sources-tinyml-auto-ml-tools/>



Edge vs. Extreme Edge (TinyML) Platforms



Source*



Source**

Edge GP-GPU	AI-enabled MCU
Jetson Orin Nano*	STM32N6**
six ARM cores	Arm® Cortex®-M55
1024 CUDA cores	ST Neural-ART accelerator
8 GB RAM	4.2 MB RAM
SSD storage support	
15 Watt	200 mW = 0.2 W
67 TOPS	600 GOPS = 0.6 TOPS
4.5 TOPS/W	3 TOPS/W
\$249	\$12.74

Further AI-enabled MCUs

Infineon
PSOC™ Edge E83

TI
TMS320F28P55x

...

* <https://www.geeky-gadgets.com/nvidia-jetson-orin-nano-mini-pc>

**05.03.2025 <https://community.st.com/t5/developer-news/stm32n6-highlights-from-the-stm32-summit/ba-p/766783>,
<https://estore.st.com/en/products/microcontrollers-microprocessors/stm32-32-bit-arm-cortex-mcus/stm32-high-performance-mcus/stm32n6-series.html>

TinyML Example: Radar Gesture Recognition

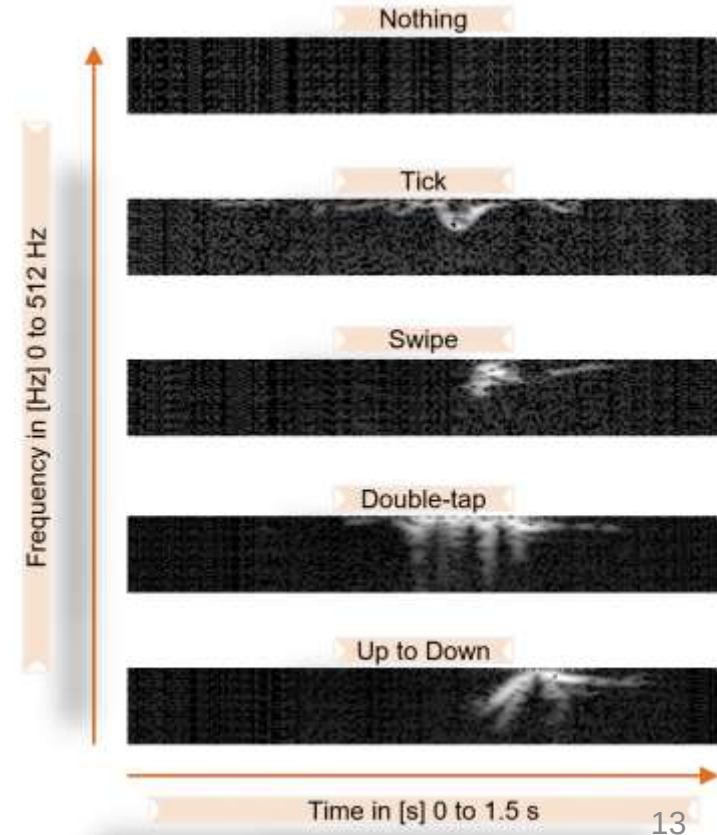
Radar spectrograms for
different hand gestures [1]

Tiny platform: Infineon XMC1302:
32 MHz CPU,
32 kB Flash, 16 kB RAM



Nothing	20	0	0	0	0
Up – Down	0	19	1	0	0
Swipe	0	0	19	1	0
Tick	0	0	1	19	0
Double Tap	0	0	0	0	20
	Nothing	Up – Down	Swipe	Tick	Double Tap

[1] Pragathi Jayaram: *Real-Time Hand Gesture classification on a MCU with Continuous Wave Radar and a Convolutional Neural Network*, Master thesis 2021

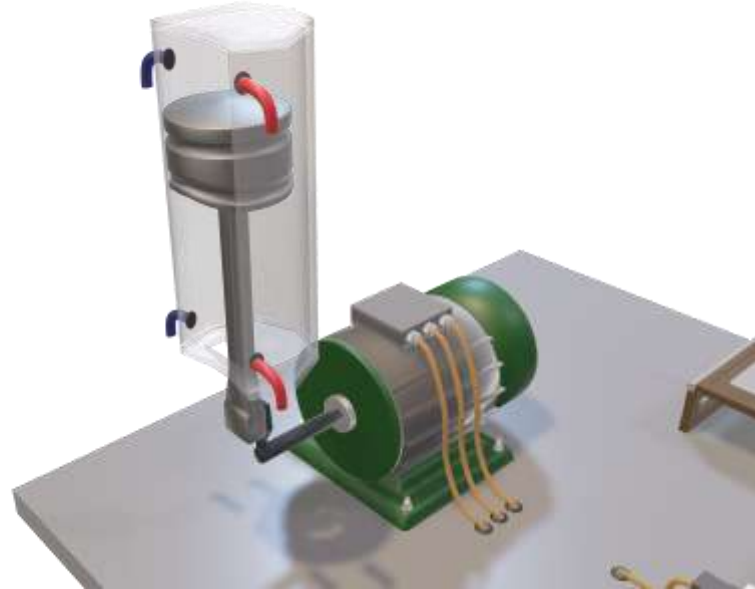


TinyML Example: AI-Enhanced Motor Control

Improve Motor Control Efficiency
using tinyML

Steven Klotz (Infineon):
AI assisted Motor Control:
From Simulation to Silicon
on a RISC-V with AI Accelerator
Thursday: 13.3:

Session 7.7 EDGE AI Case Studies 1 12:15 - 12.45



TinyMLs – As many variants as there are sensors

- **Audio**
 - Keyword Spotting (KWS) / Audio Wakeup
- **Vision**
 - Video Wakeup (Face Detection)
 - Image Classification
- **Radar**
 - Gesture Recognition
- **Accelerometer**
 - Activity Detection
 - Health care monitoring

Smart Camera Pill



<https://thecapsuleclinic.com/>

Smart safety vest recognizes accidents



<https://www.hackster.io/news/swanholm-tech-s-connected-safety-vest-is-a-wearable-tinyml-lifesaver-c472f6ac8d17>

Marvin Ogore –
The University of Rwanda, tinyML for Good



Edge AI Cholera pluggable
detection device

Designing Machine Learning Models for Tiny Embedded Systems



Some ML Terminology

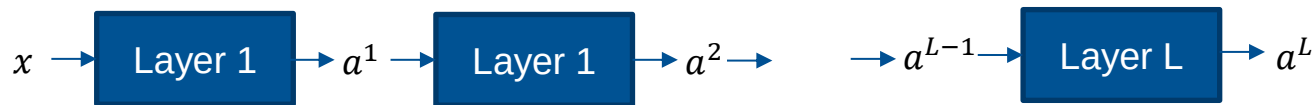
- **Deep Neural Network (DNN):** See next slide
- **ML Model:** A specific ANN for a particular application
- **Dataset:** Collection of data to train the DNN
- **Training:** Trainable parameters are tuned with the dataset
- **Inference:** A trained model is executed on an unknown new data to obtain a new label (e.g. classification result).
- **On-Device Training:** Training is done on an embedded device
- **Deployment:** A code is generated for an ANN that can be executed on the embedded device, usually only for inference.

DNN Architectures

Layered computation:

$$a^l = f^l(a^{l-1})$$

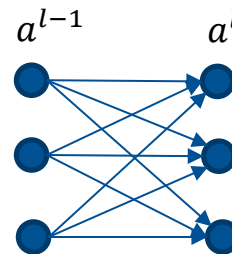
$$a^L = f^L(f^{L-1}(\dots f^1(x) \dots))$$



Example Layer: Fully-connected/Dense

- Weights: W
- Activations: a
- Activation function: f
- Bias: b

$$a^l = f^l(Wa^{l-1} + b^{l-1})$$

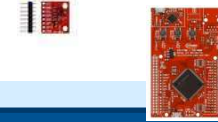


Flow - Model Generation Stage

Signals



Sensor



Collect Data

Database

Data Collection
System

Target Device &
Sensor

Preprocess Data

Data Set Collection & Labeling

Data Preparation & Preprocessing



Design a
ML Model

Neural Architecture
Search (NAS)

Design of ANN
Architecture

Preprocessed
Dataset

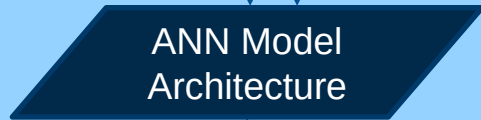
ANN Model
Architecture

Train the
ML Model

Training

Trained ANN Model

Model Evaluation



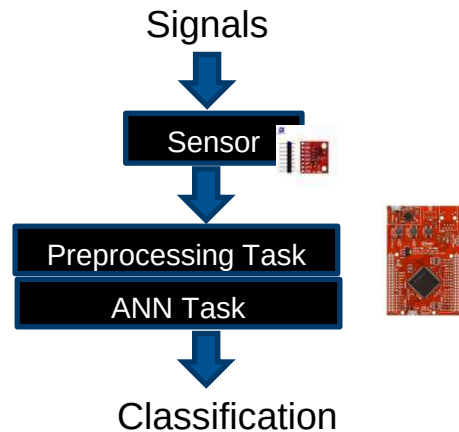
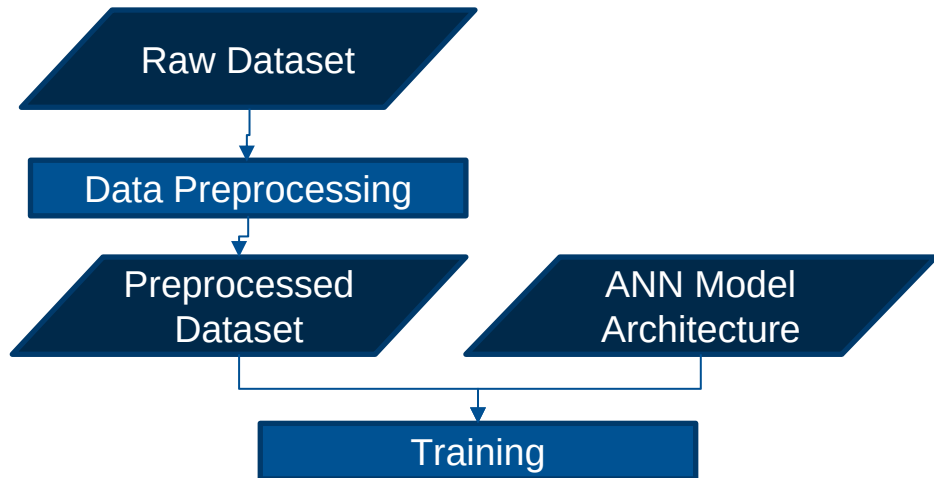
Data Set Collection & Labeling

Three sources:

- Existing Database
- Data Collection System
 - Specialized System with sensors to collect the real-world data
 - System that can produce synthetic data (e.g. by simulation)
- Embedded System with Sensor (Use the Embedded ML System itself)
- Challenges:
 - Data mismatch between collected and used sensor, e.g. different value range, resolution, missing noise and environmental conditions
 - Storage for collected data in the embedded system
 - Labels might not be automatically obtained

Data Preparation & Preprocessing

- Raw data stream from the sensor is often pre-processed before being given to a machine learning model (e.g. an ANN)
- Often DSP-type algorithms, e.g. spectral analysis (Filters, FFT, MFCC)
- Consideration: Must also run on-device for sensor data

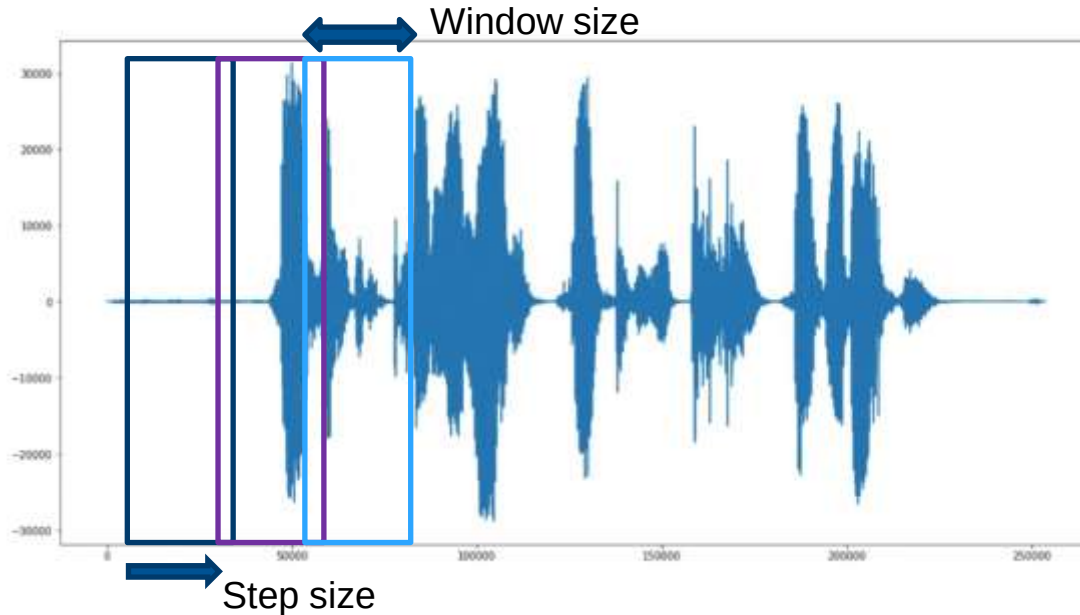


Typical pre-processing steps

- Resampling
- Window selection
- Normalization
- Filtering / Noise reduction
- Domain Change

Window Selection

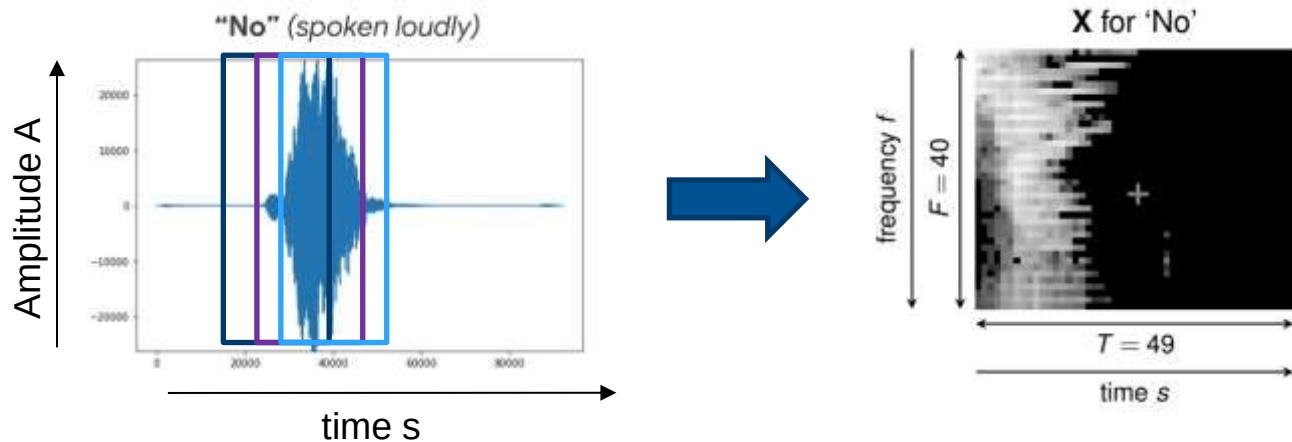
Target: Capture the pattern in the ML model inputs



Domain Change

Often: From time domain to frequency domain

CNNs often sensitive to shifts (window position) in time domain.



Source: <https://github.com/tinyMLx/courseware/blob/master/edX/slides/3-5-22.pdf>

Considerations for ML Model Architecture

- Design of the ANN model architecture must consider target system
 - ROM/RAM Memory resources (weights, activations)
 - Computational power
 - Supported datatypes: FP / integer width
 - Acceleration features (type of layers, layer configurations)
- Design of the ANN model architecture must consider application demands
 - Input resolution
 - Performance: Inferences / second
 - Model accuracy

Layer Selection

Fully connected:

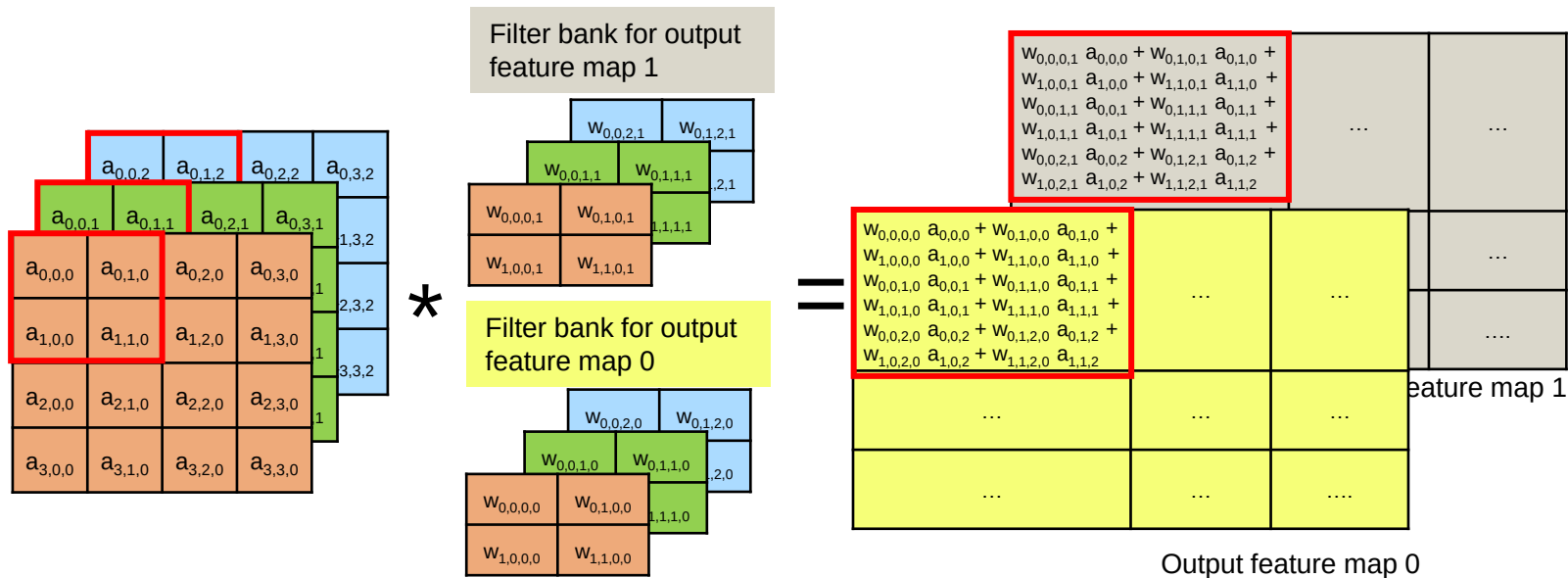
- Matrix Vector multiplication: High number of MACs
- High number of weights (ROM)

Convolutions:

- 4D Filter Bank (Memory depends on Filter Sizes)
- Convolution operation: High number of MACs
- Special Convolutions to bring down memory, MAC demand

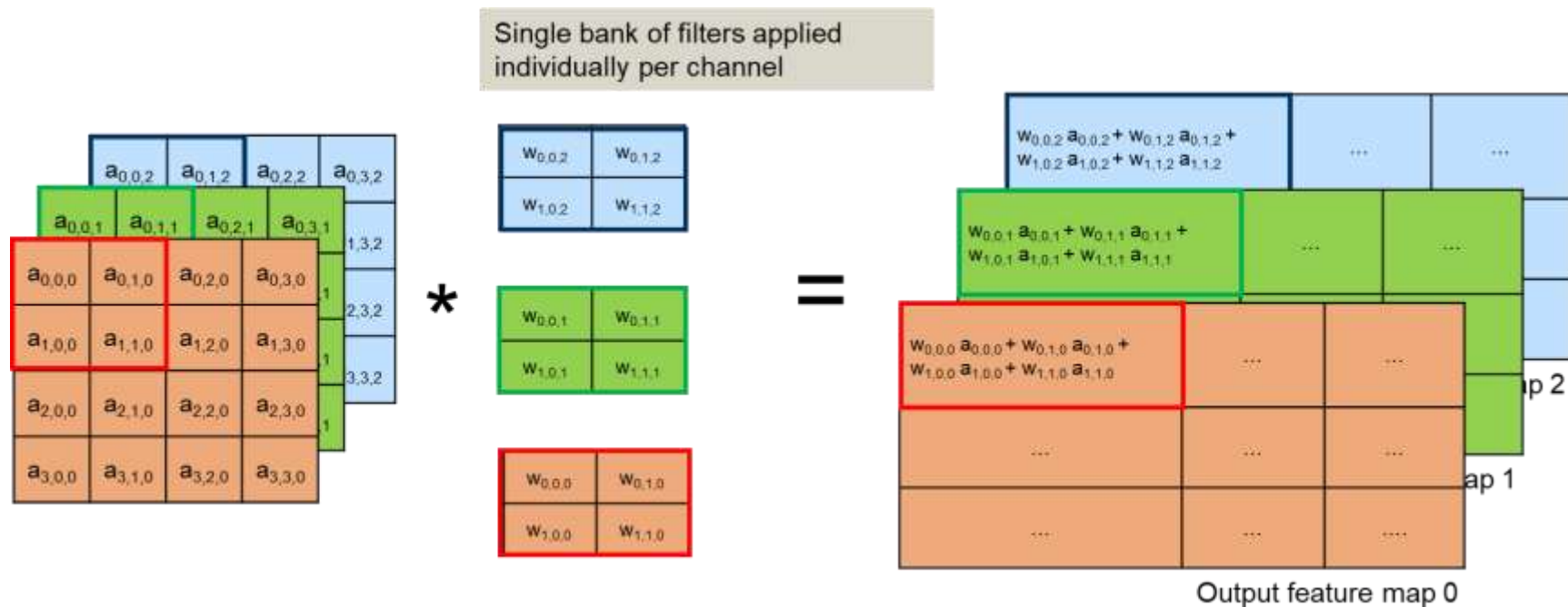
Convolution

Sliding window over input feature maps (example red and green window position)



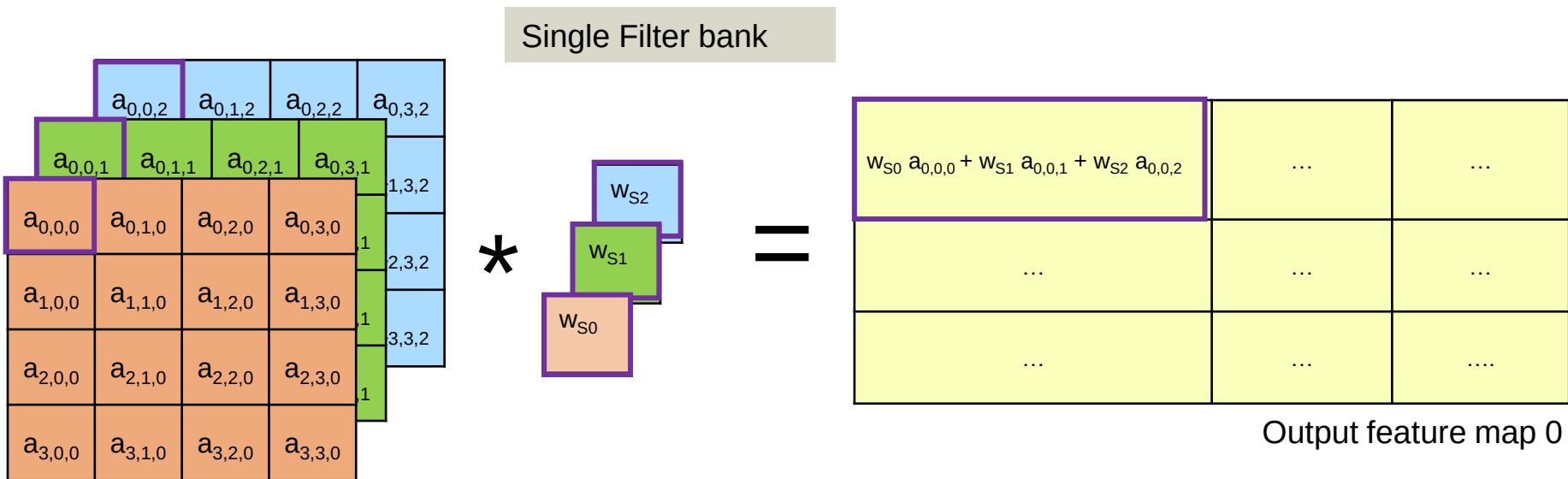
Depthwise Convolution

Single filter for each input channel (no sum over channels)



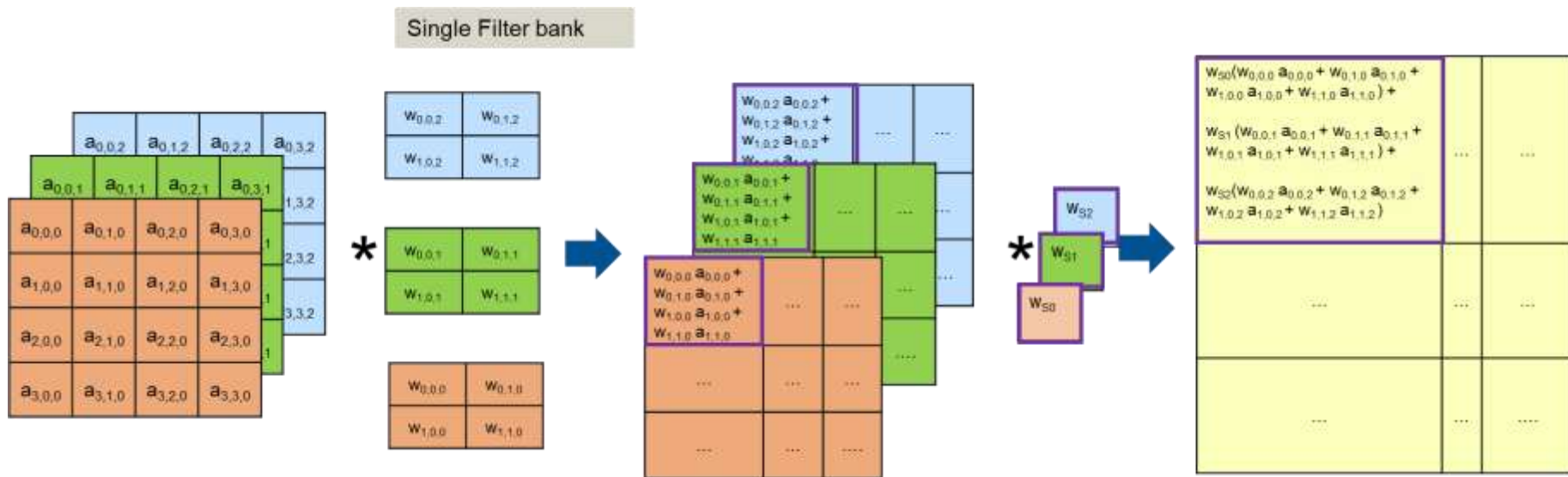
Pointwise Convolution

Filter Size 1x1: Only sum over channels



Depthwise Separable Convolution

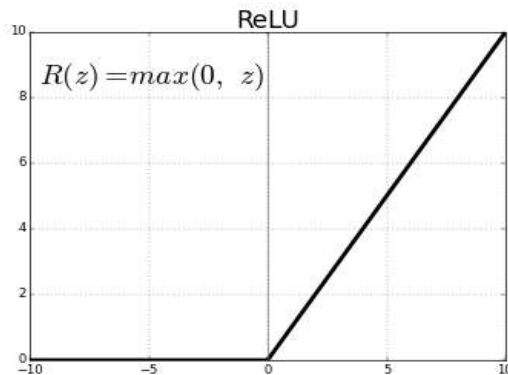
Combination of Depthwise and Pointwise Convolution



Activation Functions (Hidden Layers)

- Nonlinear functions hard to compute in HW (e.g. tanh)
 - Efficient algorithms
 - Piecewise linear approximations
- Many activations as output from hidden layers (activation function must be executed very often)
 - Often simple activation functions e.g. ReLU

```
muriscv_nn_status muriscv_nn_relu_q7(q7_t *data, const uint16_t size) {  
    for (uint16_t i = 0; i < size; i++) {  
        if (data[i] < 0) data[i] = 0;  
    }  
    return MURISCV_NN_MATH_SUCCESS;  
}
```



Activation Functions (Final Layer)

Activation function of final layer often depends on task

- **Regression:** Depends on regression problem
- **Binary Classification:** Sigmoid function maps any input to an output ranging from **0 to 1**.
- **Multi-class Classification:** Softmax converts a vector of real values to a vector of categorical/class probabilities. The elements of the output are in range $(0, 1)$ and sum to 1.

Model Types

- Feed-forward DNN: Several dense layer
- CNN: Combination of Conv layers + Pooling for feature extraction followed by Dense Layers for classification
- Auto-Encoder: Encoder-Decoder Architecture
- Recursive Neural Network (RNN): Feedback paths inside the NN (makes use of past information)
- Transformers: Special Attention Layers
- Many special model types with special layers: ResNET, ...

Model Definition

ML Frameworks: PyTorch / Tensor Flow (Keras)

```
# Step 1: Define the keras model
model1 = tf.keras.Sequential()
model1.add(tf.keras.layers.Dense(10, activation='relu', input_shape=(2,)))
model1.add(tf.keras.layers.Dense(20, activation='relu'))
model1.add(tf.keras.layers.Dense(20, activation='relu'))
# model1.add(tf.keras.layers.Dense(3, activation='softplus'))
model1.add(tf.keras.layers.Dense(3, activation='softmax'))
```

Model Evaluation

- **Model Training Target:**

Minimize error on validation set with the aid of the training set

- Make the error for the samples in the training set small
- Make the gap between the training and validation error small

- **Model Evaluation Target:** Does the model perform well for the given embedded task?

Confusion Matrix (Binary Classification)

Binary classification with two classes 0/1 (positive/negative):

	NN predicts class 0 (positive)	NN predicts class 1 (negative)
Labeled class 0 (positive)	Nr. of true positives (<i>tp</i>)	Nr. of false negatives (<i>fn</i>)
Labeled class 1 (negative)	Nr. of false positives (<i>fp</i>)	Nr. of true negatives (<i>tn</i>)

Model Evaluation for Binary Classification

Accuracy:

$$a = \frac{tp+tn}{tp+tn+fp+fn}$$

How many samples were classified correctly?

Precision:

$$p = \frac{tp}{tp+fp}$$

How many of the samples classified as positives were really positive?

Recall:

$$r = \frac{tp}{tp+fn}$$

How many of the positive samples were found over the total amount of positive samples?

F1 score:

$$F1 = 2 \frac{p \cdot r}{p+r}$$

Combined value for precision and recall.

Confusion matrix

Prediction

Positive Negative

**Ground
Truth**

Positive

tp

fn

Negative

fp

tn

Case Study: Anomaly Detection (1/6)



Machine

Vibration
Sensor



NN
Model



ALARM

Positive (Alarm): Machine has an anomaly: Conduct predictive maintenance
Negative: Machine has no anomaly

Test Set:

- Negative samples with no anomaly: 1000
- Positive samples with anomaly: 20

We have three trained NN candidates: **NN1**, **NN2** and **NN3**

Case Study: Anomaly Detection (2/6)

NN1 prediction

	Positive	Negative
Positive	1	19
Negative	2	998

Accuracy: $a = \frac{1+998}{1+998+19+2} = 0,98$

Precision: $p = \frac{1}{1+2} = 0,33$

Recall: $r = \frac{1}{1+19} = 0,05$

F1 score: $F1 = 2 \frac{0,05 \cdot 0,33}{0,05+0,33} = 0,09$

NN2 prediction

	Positive	Negative
Positive	19	1
Negative	200	800

Accuracy: $a = \frac{19+800}{19+800+1+200} = 0,80$

Precision: $p = \frac{19}{19+200} = 0,09$

Recall: $r = \frac{19}{18+2} = 0,95$

F1 score: $F1 = 2 \frac{0,95 \cdot 0,09}{0,95+0,09} = 0,16$

Case Study: Anomaly Detection (3/6)

Accuracy of **NN1** and **NN2** has little meaning (high for both NNs)

NN1:

Medium precision: Many alarms are false

Low recall: Most anomalies are overlooked

➡ Low F1 score

NN2:

Low precision: Most alarms are false

High recall: Most anomalies are detected

➡ Low F1 score

Case Study: Anomaly Detection (4/6)

		NN3 prediction	
		Positive	Negative
Positive	18	2	
Negative	40	960	

Accuracy: $a = \frac{18+960}{18+960+40+2} = 0,96$

Precision: $p = \frac{18}{18+40} = 0,31$

Recall: $r = \frac{18}{18+2} = 0,9$

F1 score: $F1 = 2 \frac{0,9 \cdot 0,31}{0,9+0,31} = 0,46$

Accuracy of NN3 lower than NN1

NN3:

Average precision:

Many alarms are still false

High recall:

Most anomalies are detected

➡ Average F1 score

Is NN3 suitable for the task?

Case Study: Anomaly Detection (5/6)

- On average one anomaly after 30 days of machine operation
- One monitoring sample from vibration sensor per minute
- On average: $30 * 24 * 60$ negative samples : 43.200 negative samples

Negative samples: $tn = 960$ $fp = 40$

- *Probability of false alarm for false positive sample: $P(fp) = 40/1000$*

„Pi-Mal-Daumen“ (Pi-times-thumb - German saying for rough estimation):

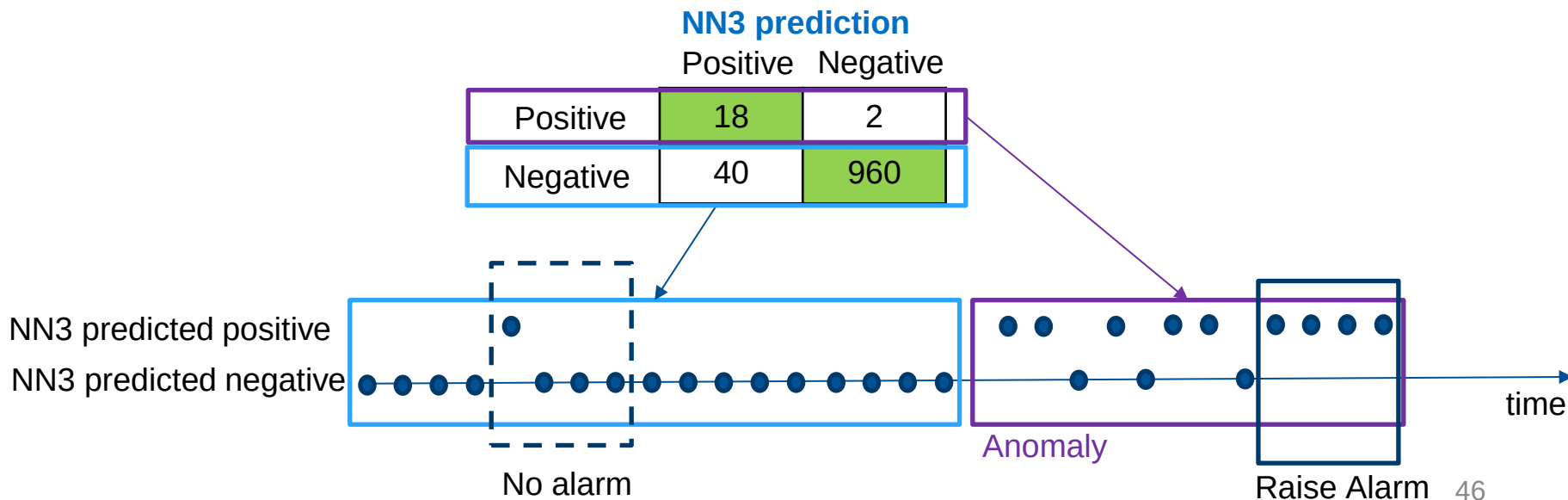
- On average we have $(40/1000) * 43.200 = 40 * 43.2 = 1.728$ false alarms within the 30 days

Solution?

Case Study: Anomaly Detection (6/6)

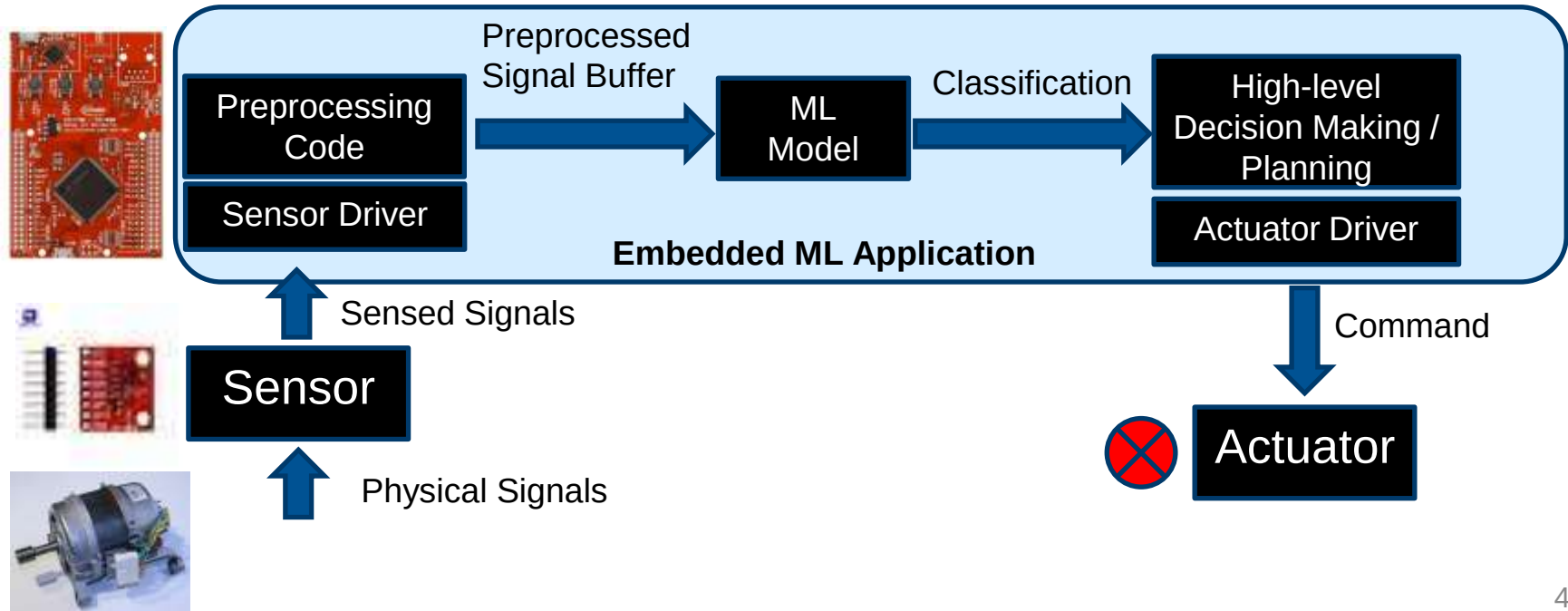
Possible solution:

- Use temporal correlation: Several samples measured during anomaly
- High-level decision logic: to raise alarm, many consecutive samples needs to be positive (with high probability).



ML Model is Embedded into Application

Typical SW Structure of an Embedded Machine Learning Application
Needs to be considered for training ML model



Case Study 2: Anomaly Detection (1/2)



Machine

Vibration
Sensor



NN
Model



ALARM

Positive (Alarm): Machine has an anomaly: Conduct predictive maintenance
Negative: Machine has no anomaly

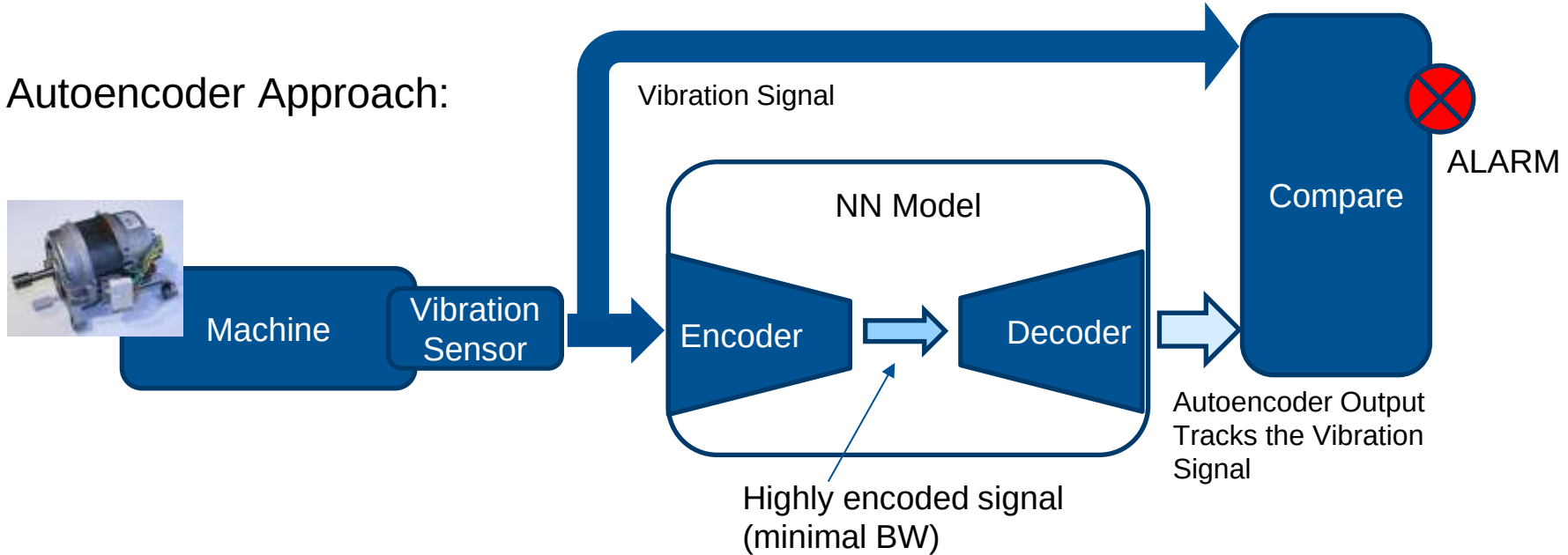
Test Set:

- Negative samples with no anomaly: 1000
- Positive samples with anomaly: **NONE**

Can we still solve the task in a data-driven way?

Case Study 2: Anomaly Detection (2/2)

Autoencoder Approach:

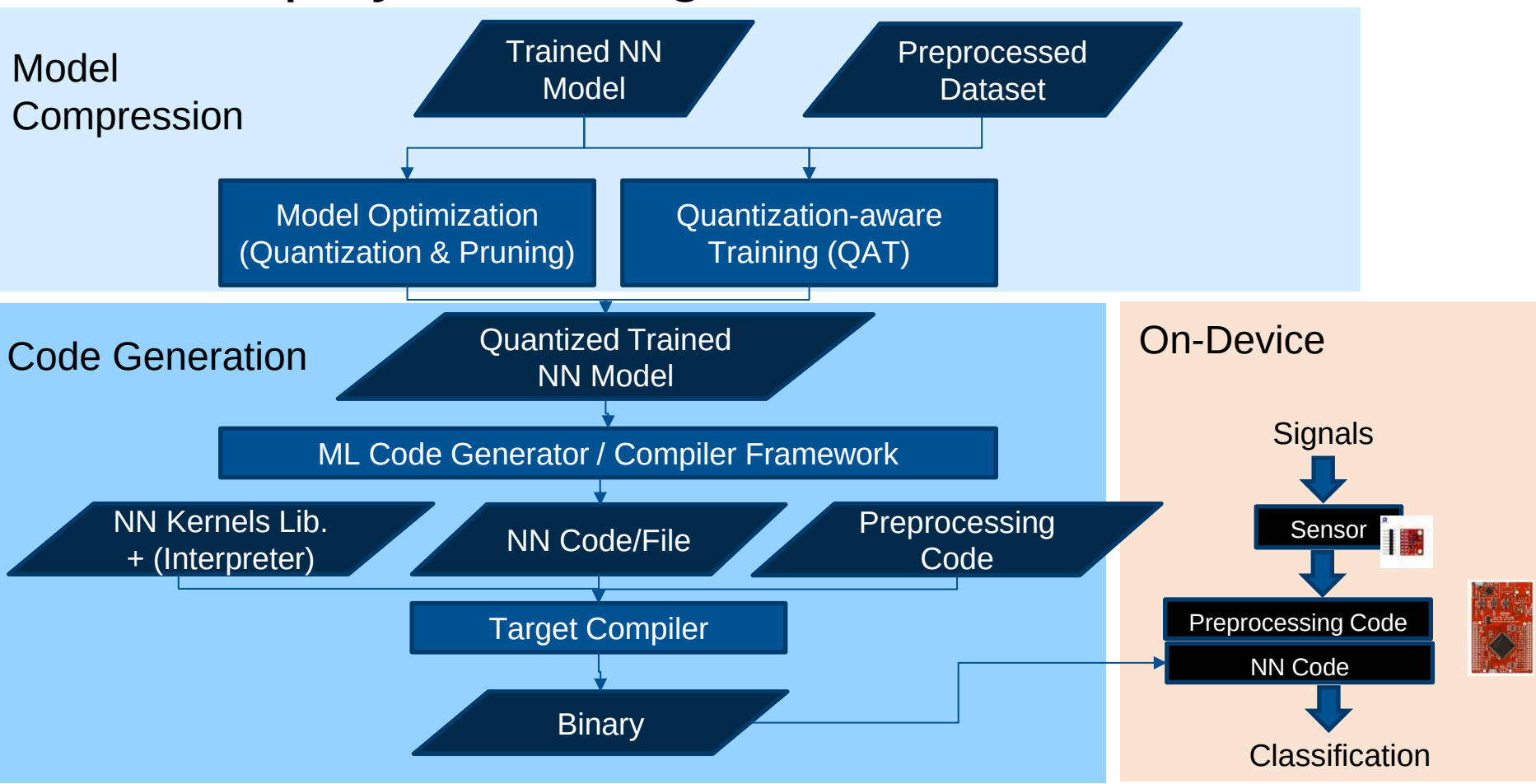


- Autoencoder trained so that its output follows the vibration signal based on encoded signal for normal operation.
- Anomaly: Autoencoder output cannot follow vibration signal.

Deploying Machine Learning Models on Tiny Embedded Systems



Flow – Deployment Stage



Tasks of Code Generation

Kernel Generation and Optimization:

- Generate kernel codes that implement the computation of a layer
- Optimize kernels (e.g. fusing layers) for efficient execution on the target HW

Storage for read only data:

- Decide on format how to store weight tensors
- Plan memory to store weight tensors and layer configurations

Storage for read-write data:

- Decide on format how to store activation tensors
- Plan memory to store the activation tensors (memory reuse)

Model Graph Generation / Runtime Setup:

- Layer Scheduling
- Generate code or include runtime that executes the model graph (calls layer kernels on the tensors)

Open Source ML Deployment Frameworks

Tensor Flow Lite / TFLM (Google) [<https://www.tensorflow.org/lite/microcontrollers>]

- Offers a runtime (interpreter) that can execute a quantized model graph
- Quantized model graph is stored in a flatbuffer file (.tflite)
- Provides reference (hardcoded) kernel library that the interpreter calls
- Usually kernel library is replaced with efficient inference library (e.g. for ARM: CMSIS-NN)

Apache TVM / MicroTVM [<https://tvm.apache.org/>]

- Compiler-based optimization and kernel generation based on high- and low-level IR
- Offers several backends for different targets either interpreter-based or static code generation
- Support auto-tuning for HW-specific optimizations and HW accelerator support via BYOC interface

(Tiny)IREE [<https://github.com/iree-org/iree>]

- Based on LLVM's MLIR (Multi Level IR) Project
- Highly flexible and modular
- Lightweight runtime or bytecode interpreter

Commercial Deployment Flows

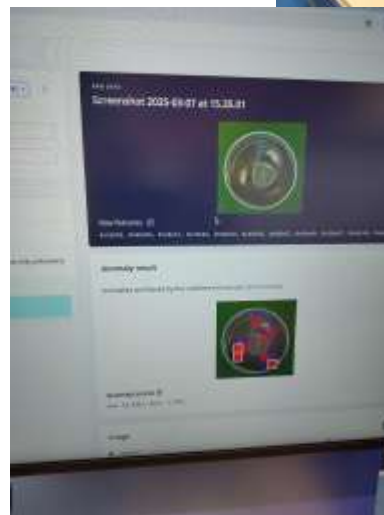
Incomplete list!

MCU-Vendor deployment tools:

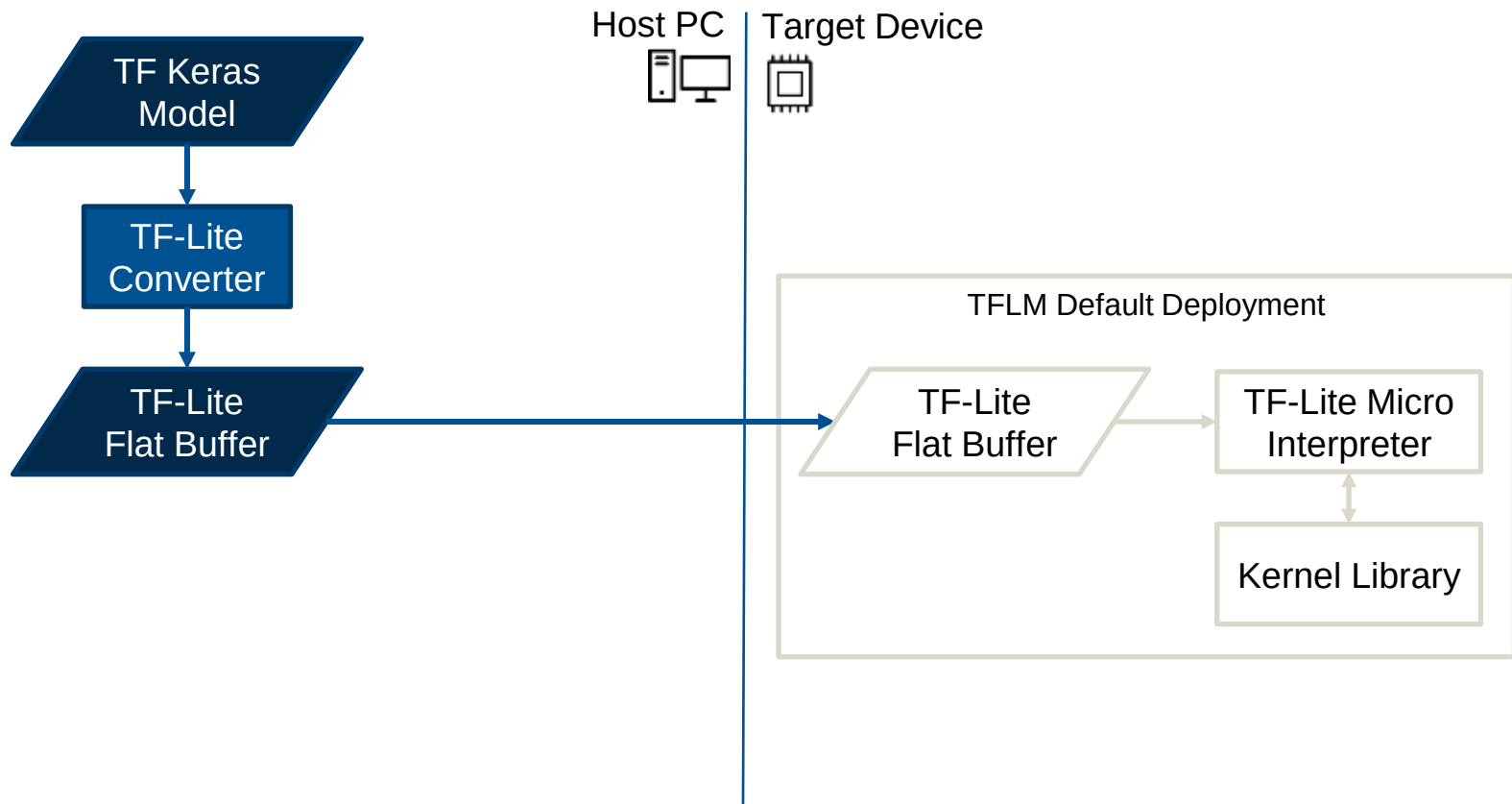
- Infineon's ModusToolbox™ ML platform
- ST X-CUBE-AI
- ...

AutoESL tools:

- Imagimob AI
- SensiML
- Edge Impulse
- ...



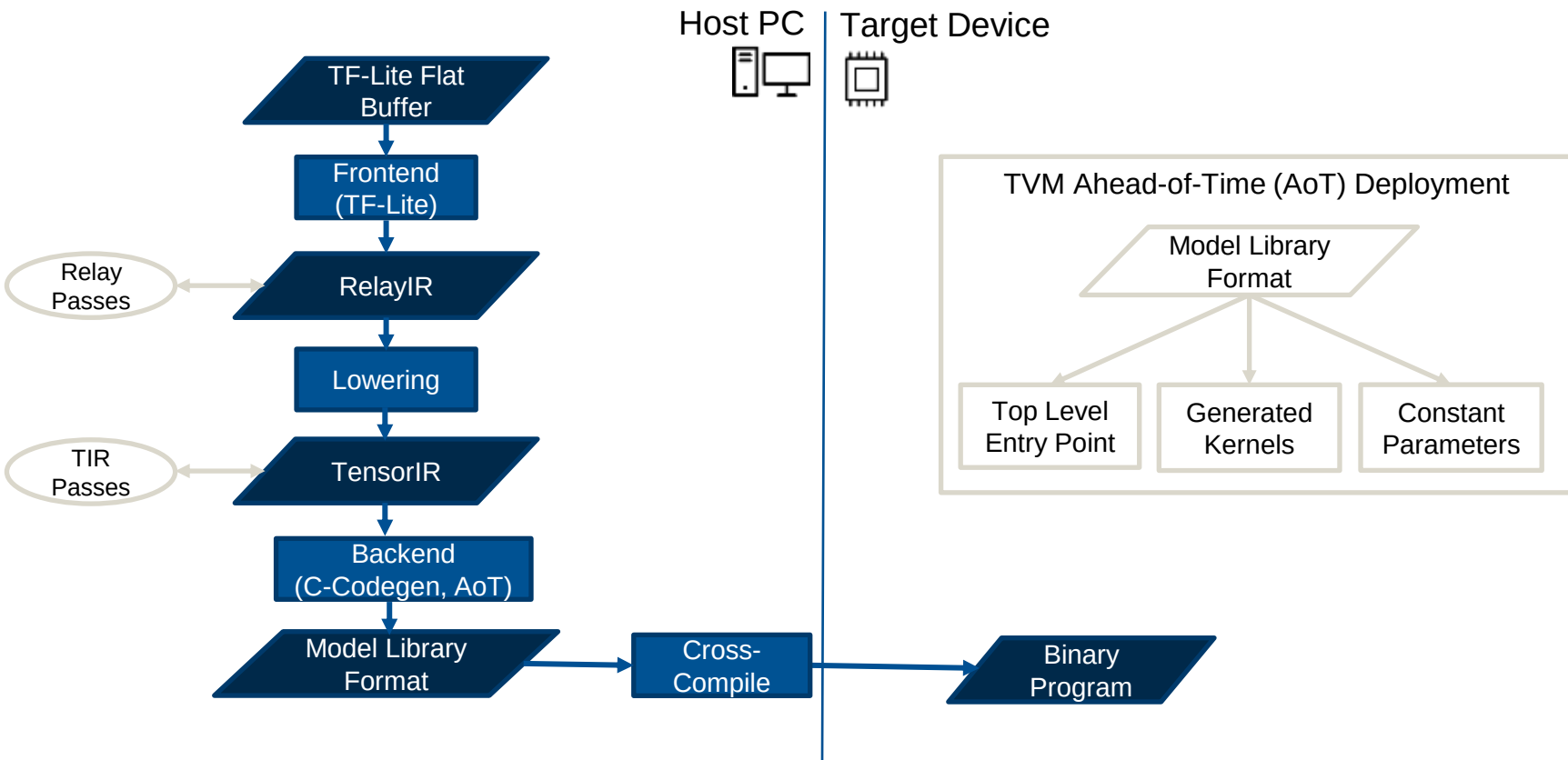
Code Generation Flow (TFLM) (1/2)



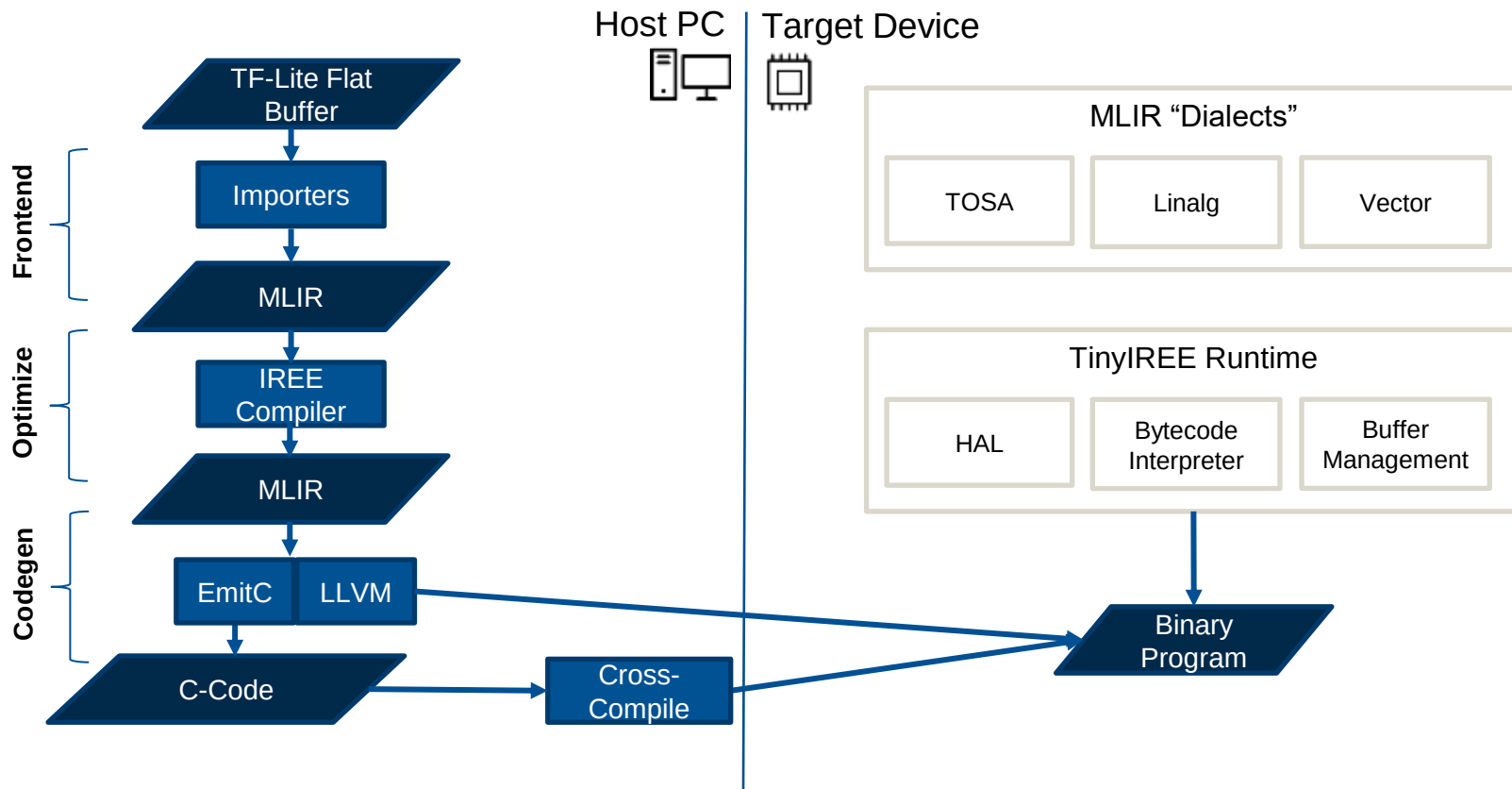
Code Generation Flow (TFLM) (2/2)

- TensorFlow / Keras models are converted into TF-Lite format (.tflite)
 - Includes automated quantization
 - Loads model structure
 - Does memory planning
 - Calls into kernel library

Code Generation Flow (TVM)



Code Generation Flow (TinyIREE)



TinyML Tutorial Overview

14:00 Daniel Müller-Gritschneider:
Introduction to TinyML

14:45 Break and Q&A

15:00 Marcus Rüb:
Introduction to Model Compression Techniques

15:30 Break and Q&A

15:45 Philipp van Kempen:
Showcase: TinyML Keyword Spotting (KWS)

16:30 Q&A and Wrap-up

Please feel free to
always ask in
between!

